UTM

UNIVERSITI TEKNOLOGI MALAYSIA

WEB TECHNOLOGY (SECJ3483)

SEM 2 (2025/2026)

GROUP LAB ASSIGNMENT REPORT

CASE STUDY: SECURE PERSON BMI FULL-STACK WEB APPLICATION

LECTURER'S NAME:

DR NORIZAM BIN KATMON

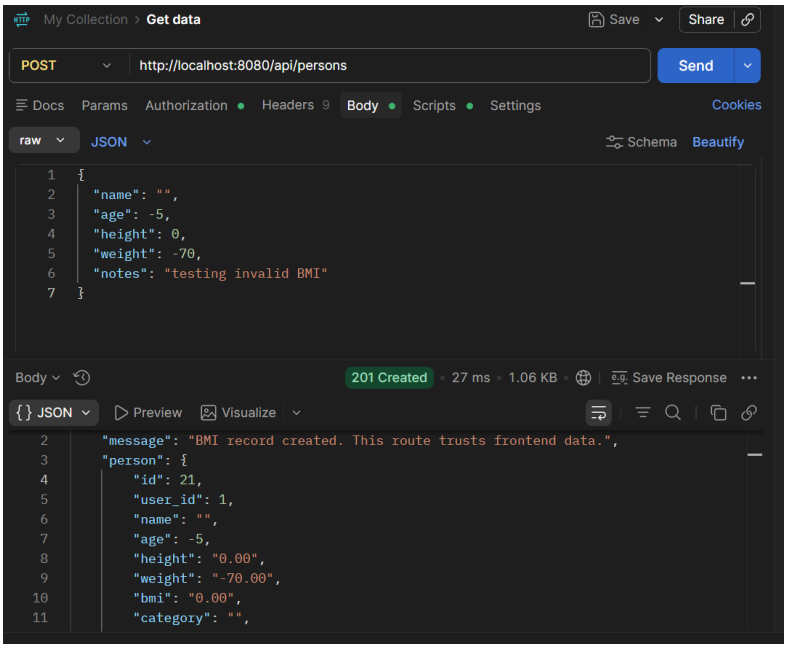
SECTION: 01

PREPARED BY :

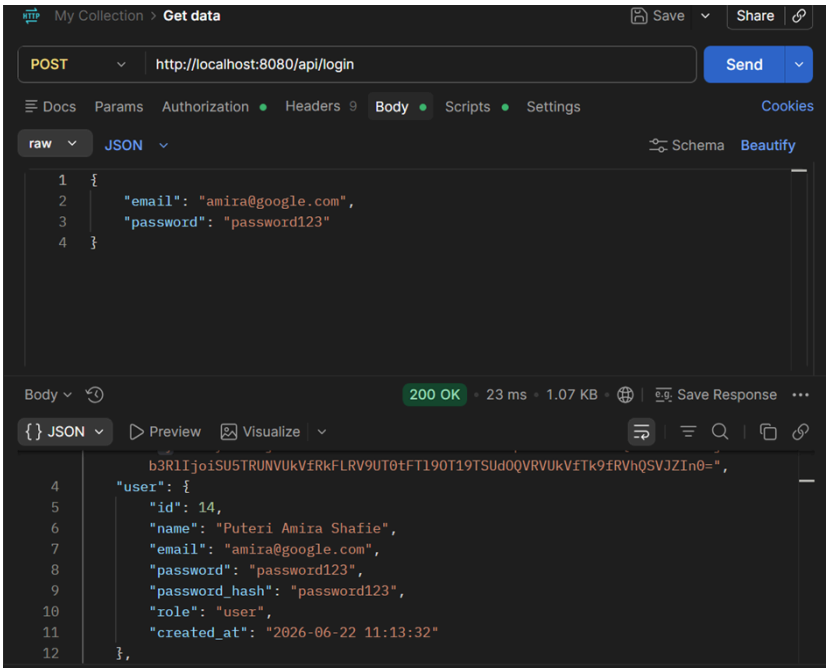
NO.	NAME	MATRIC NO
1.	CHUAH HUI WEN	A23CS0219
2.	ANJUM SIDDIQUA TANVEER SIDDIQUI	A23CS0289
3.	THANG WEI JIE	A23CS0280

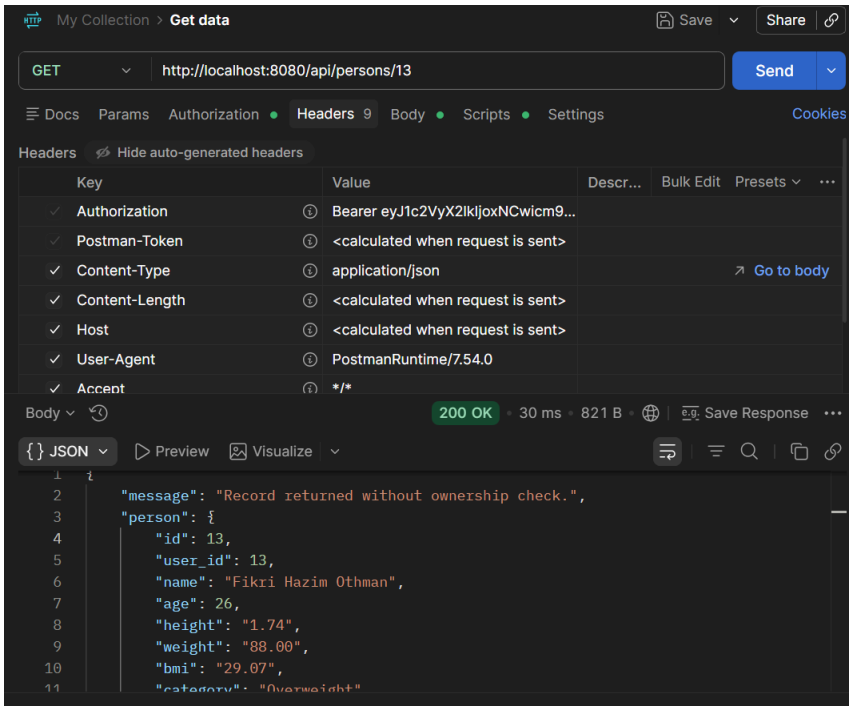
1. Screen Snapshot Before Security Fixes

Mission 1:

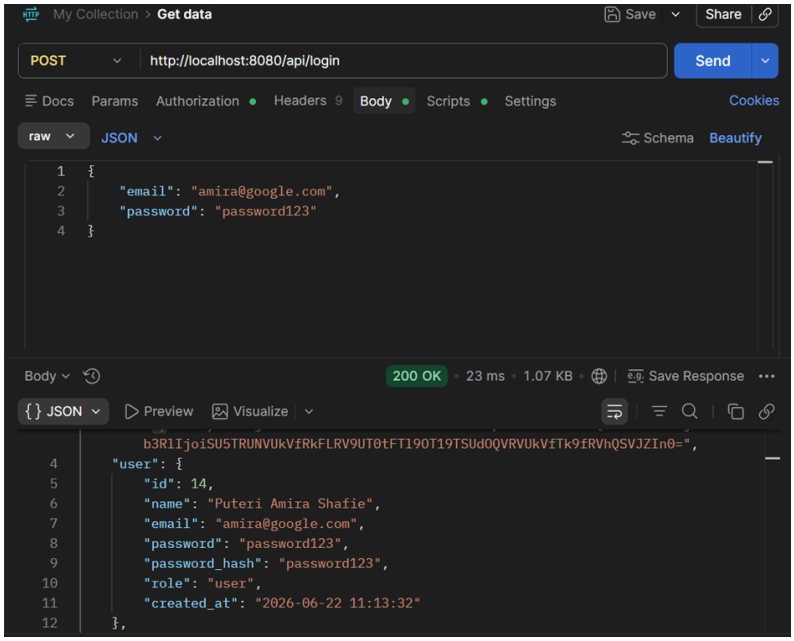


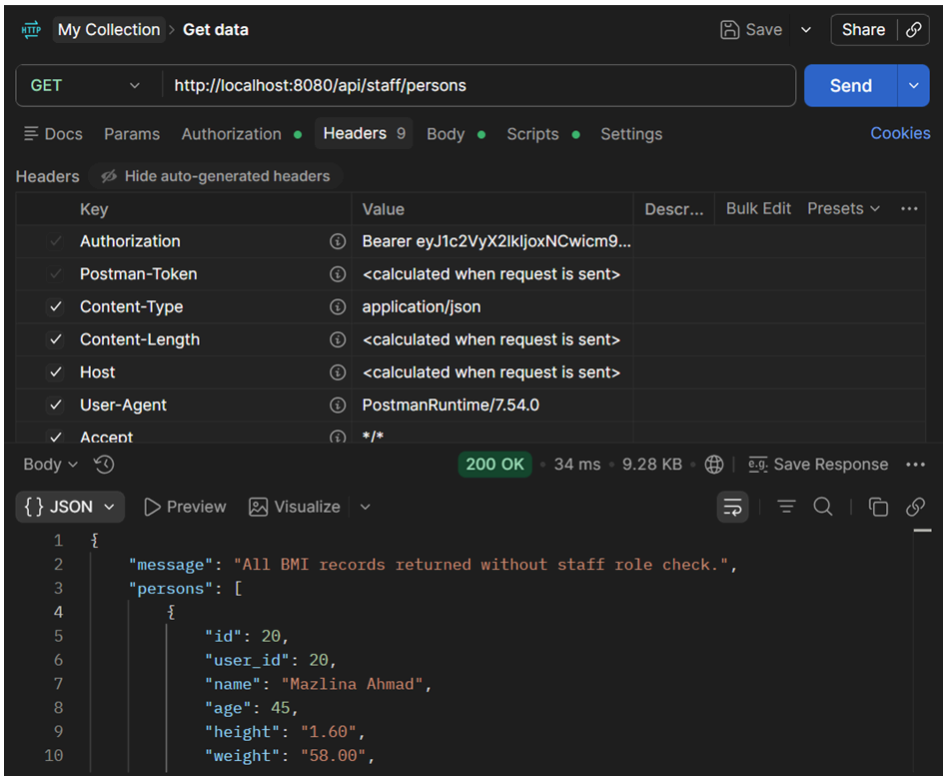
Mission 2:

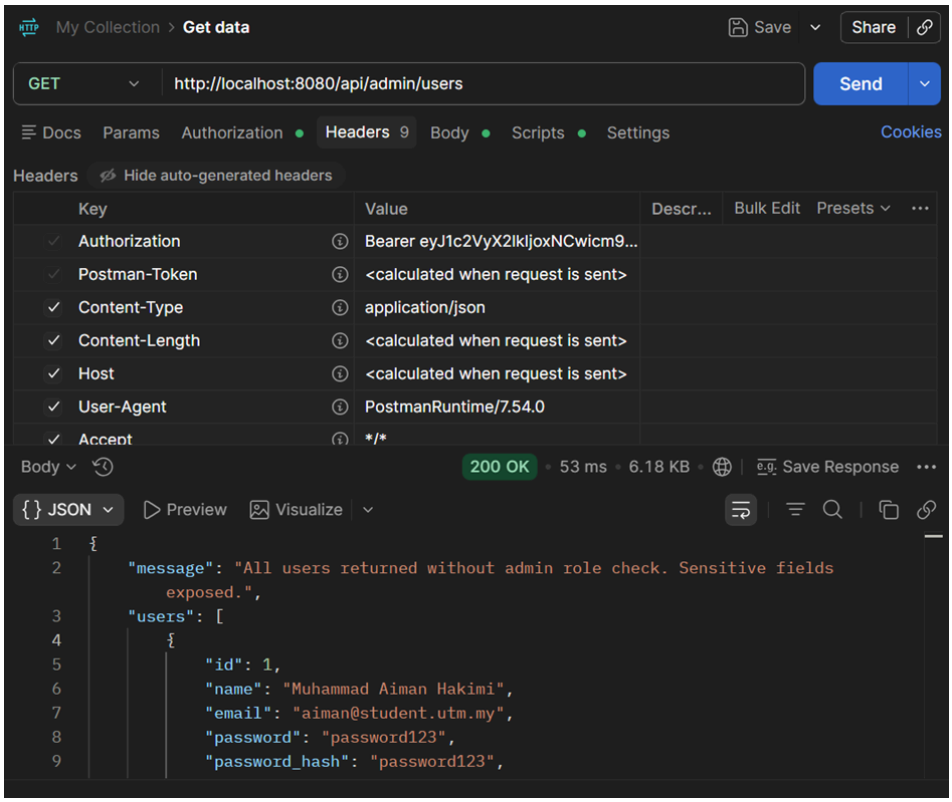



My Collection > Get data
Save
Share
GET
http://localhost:8080/api/persons/13
Send
Docs
Params
Authorization
Headers 9
Body
Scripts
Settings
Cookies
Headers
Hide auto-generated headers
Key
Value
Descr...
Bulk Edit
Presets
Authorization
Bearer eyJ1c2VyX2lkIjoxNCwicm9...
Postman-Token
<calculated when request is sent>
Content-Type
application/json
Go to body
Content-Length
<calculated when request is sent>
Host
<calculated when request is sent>
User-Agent
PostmanRuntime/7.54.0
Accept
/
Body
200 OK
30 ms
821 B
Save Response
JSON
Preview
Visualize
{
"message": "Record returned without ownership check.",
"person": {
"id": 13,
"user_id": 13,
"name": "Fikri Hazim Othman",
"age": 26,
"height": "1.74",
"weight": "88.00",
"bmi": "29.07",

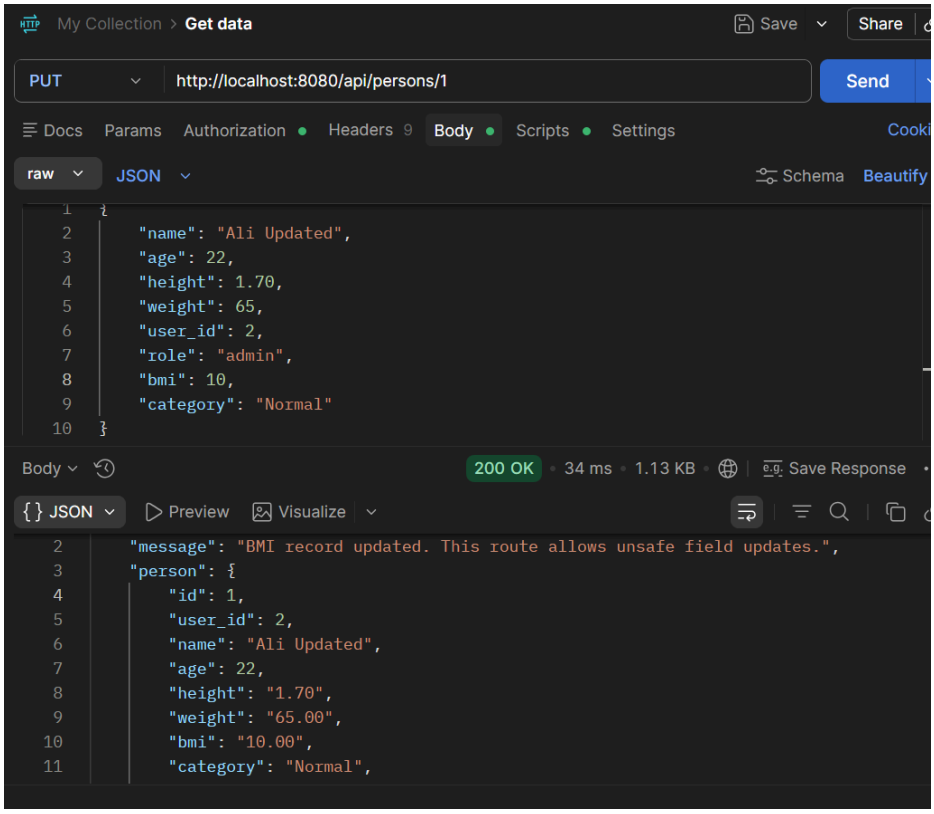
Mission 3:


My Collection > Get data
Save
Share
POST
http://localhost:8080/api/login
Send
Docs
Params
Authorization
Headers 9
Body
Scripts
Settings
Cookies
raw
JSON
Schema
Beautify
{
"email": "amira@google.com",
"password": "password123"
}
Body
200 OK
23 ms
1.07 KB
Save Response
JSON
Preview
Visualize
b3RlIjoiSU5TRUNVUkVfRkFLRV9UT0tFT19OT19TSUdOQVRVUkVfTk9fRVhQSVJZIn0=",
"user": {
"id": 14,
"name": "Puteri Amira Shafie",
"email": "amira@google.com",
"password": "password123",
"password_hash": "password123",
"role": "user",
"created_at": "2026-06-22 11:13:32"
},

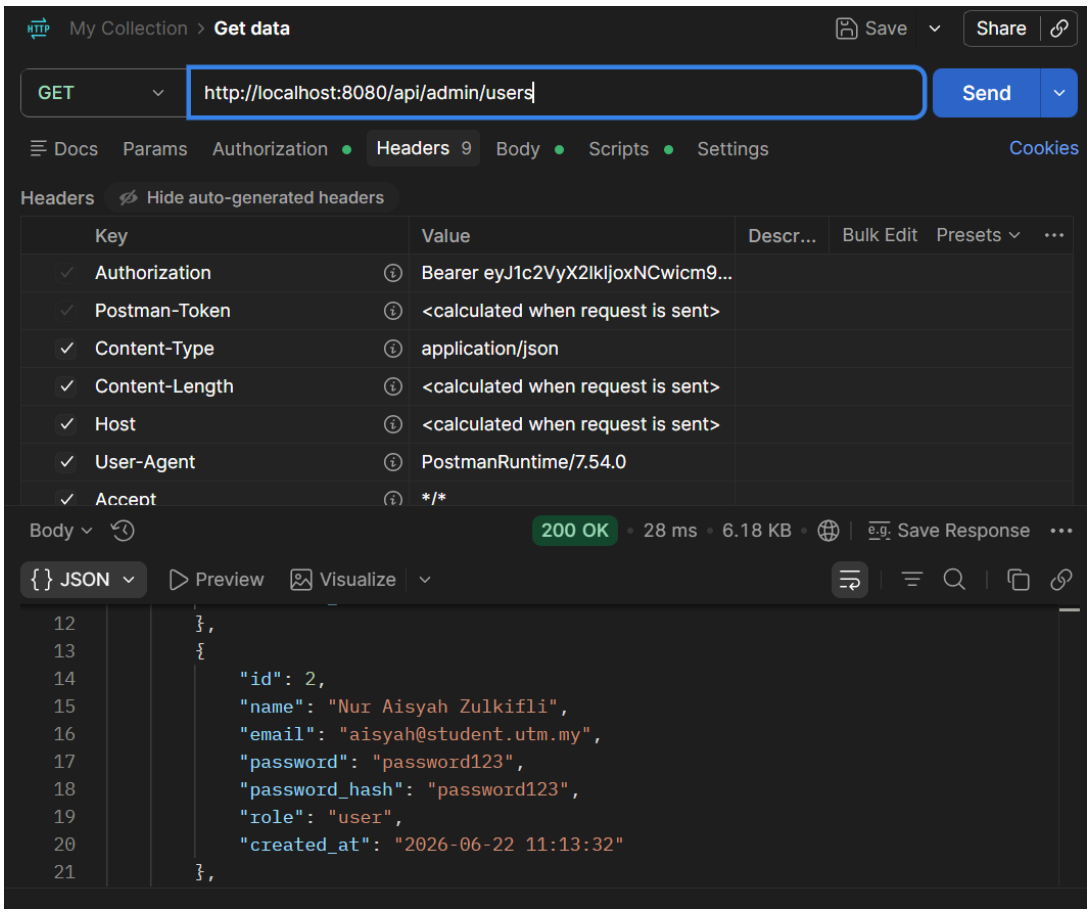

My Collection
Get data
Save
Share
GET
http://localhost:8080/api/staff/persons
Send
Docs
Params
Authorization
Headers 9
Body
Scripts
Settings
Cookies
Headers
Hide auto-generated headers
Key
Value
Descr...
Bulk Edit
Presets
Authorization
Bearer eyJ1c2VyX2lkIjoxNCwicm9...
Postman-Token
<calculated when request is sent>
Content-Type
application/json
Content-Length
<calculated when request is sent>
Host
<calculated when request is sent>
User-Agent
PostmanRuntime/7.54.0
Accept
/
Body
200 OK
34 ms
9.28 KB
Save Response
JSON
Preview
Visualize
{
"message": "All BMI records returned without staff role check.",
"persons": [
{
"id": 20,
"user_id": 20,
"name": "Mazlina Ahmad",
"age": 45,
"height": "1.60",
"weight": "58.00",


My Collection
Get data
Save
Share
GET
http://localhost:8080/api/admin/users
Send
Docs
Params
Authorization
Headers 9
Body
Scripts
Settings
Cookies
Headers
Hide auto-generated headers
Key
Value
Descr...
Bulk Edit
Presets
Authorization
Bearer eyJ1c2VyX2lkIjoxNCwicm9...
Postman-Token
<calculated when request is sent>
Content-Type
application/json
Content-Length
<calculated when request is sent>
Host
<calculated when request is sent>
User-Agent
PostmanRuntime/7.54.0
Accept
/
Body
200 OK
53 ms
6.18 KB
Save Response
JSON
Preview
Visualize
{
"message": "All users returned without admin role check. Sensitive fields exposed.",
"users": [
{
"id": 1,
"name": "Muhammad Aiman Hakimi",
"email": "aiman@student.utm.my",
"password": "password123",
"password_hash": "password123",

Mission 4:



Mission 5:



Mission 6:

Add BMI Record

Insecure starter: BMI and category are calculated in frontend and sent to backend. Students should question this design.

Name	Age	Height (meters)	Weight (kg)
Ali	21	1.70	65

Notes

```
<img src=x onerror="alert('XSS')">
```

Frontend-calculated BMI	Frontend-calculated Category
22.49	Normal

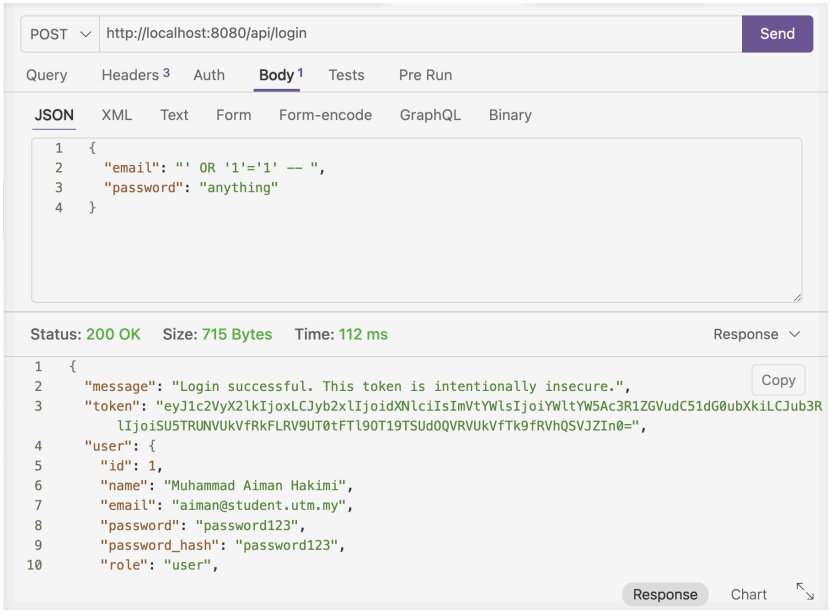
▾ Insecure payload preview

SQLSTATE[42000]: Syntax error or access violation: 1064 You have an error in your SQL syntax; check the manual that corresponds to your MariaDB server version for the right syntax to use near 'alert("XSS")'>')' at line 2

Raw API response


```
{
  "ok": false,
  "status": 500,
  "data": {
    "error": "SQLSTATE[42000]: Syntax error or access violation: 1064 You have an error in your SQL syntax; check the manual that
corresponds to your MariaDB server version for the right syntax to use near 'alert(\"XSS\")'>')' at line 2",
    "file": "C:\\Users\\USER\\Desktop\\group-lab-assignment-security-2-max-4-members-group-AStanveer\\person-bmi-slim-insecure-backend\
\public\\index.php",
    "line": 299
  }
}
```


Mission 7:



2. Security investigation summary

The starter application was intentionally shipped insecure for this lab: it trusted frontend-calculated values, used an unsigned fake token, ran raw string-interpolated SQL, and applied no authorization checks. The team used Postman/Thunder Client to probe each endpoint and confirm each weakness before fixing it.

Investigation Table

Test No	What I Tested	Expected Secure Behavior	Actual Result	Is It a Weakness?	Why?
1	Add BMI with invalid data	Backend should reject invalid data	201 Created — record saved with age:-5, height:0.00, weight:-70.00, bmi:0.00	Yes	No backend validation; client-supplied values went straight into the INSERT
2	Access another user's BMI record	Backend should deny access	200 OK — "Record returned without ownership check.", full record for user_id:13 returned	Yes	No comparison between the requester's identity and the record's user_id (BOLA)
3	Access staff route as normal user	Backend should deny access	200 OK — "All BMI records returned without staff role check.", full dataset (9.28 KB) returned	Yes	No role check on the route handler before querying (BFLA)
4	Access admin route as normal user	Backend should deny access	200 OK — "All users returned without admin role check. Sensitive fields exposed.", including password and password_hash in plaintext	Yes	No role check; SELECT * exposes every column including credentials
5	Submit XSS payload in notes	Payload should display as text	500 Internal Server Error — the unescaped apostrophe in the payload broke the raw SQL string (SQLSTATE[42000]: ... syntax error ... near 'alert(\"XSS\")'), and the error response leaked the full server file path and line number	Yes	Two compounding bugs: (a) notes were interpolated into SQL unescaped, and (b) the error handler exposed internal stack details. Even when the insert succeeds, v-html would have executed the payload on render
6	Check API response	Password/hash should not be visible	Visible in plaintext on /api/login, /api/register, /api/admin/users (e.g. "password_hash": "password123")	Yes	Routes used SELECT * and returned the full row, including credential fields
7	Try SQL Injection input	Input should be treated as data	200 OK — "Login successful. This token is intentionally insecure."; authentication bypassed, returned a valid (insecure) token for the first matching user	Yes	Query built with raw string concatenation, no prepared statement
8	Try to update protected fields	Backend should reject or ignore protected fields	200 OK — "This route allows unsafe field updates."; user_id, role, and bmi were all overwritten as sent	Yes	No field whitelist on the UPDATE; every key in the request body was blindly written to the query

3. Weaknesses classification table

Weakness Found	Frontend / Backend / API / Database	Security Category	Possible Risk
Negative weight accepted	Backend	Missing backend validation	Invalid data stored
User accesses another BMI record	API / Backend	Broken Object Level Authorization	Privacy violation
Normal user accesses admin route	API / Backend	Broken Function Level Authorization	Unauthorized admin action
API returns password_hash	API / Backend	Sensitive data exposure	Credential risk
XSS payload executes	Frontend	XSS	Browser script execution
SQL Injection input affects query	Backend / Database	SQL Injection	Unauthorized data access
User changes user_id or role	API / Backend	Mass assignment	Privilege or ownership manipulation

4. Fixes implemented

All backend fixes live in person-bmi-slim-insecure-backend/public/index.php unless noted; the frontend fix is in person-bmi-vue-cli-insecure-starter/src/components/BmiCard.vue.

Fix	What changed	How it addresses the weakness
Fix 1 — Backend validation	Backend does not validate	Closes Test 1 — invalid data can no longer be stored regardless of what the frontend sends
Fix 2 — Backend BMI calculation	Added calculateBmi() / getBmiCategory(); bmi and category are computed server-side from height/weight and the client-supplied bmi/category are discarded	Frontend-calculated BMI can no longer be spoofed
Fix 3 — Password hashing	sql/seed.sql now stores bcrypt hashes ($2y$10$...) in password_hash; plaintext password column dropped from seed inserts	Reduces blast radius of a database leak — credentials are no longer recoverable in plaintext from seed data
Fix 4 — Prepared statements	register/login queries rewritten with $pdo->prepare() + bound parameters instead of string concatenation	Closes Test 7 — SQL Injection payloads are treated as literal data, not SQL syntax
Fix 5 — Real signed JWT	Replaced base64-only fake token with firebase/php-jwt, HS256-signed, including iat and exp (1-hour expiry)	Tokens can no longer be forged or edited client-side; they expire
Fix 6 — JWT check on all protected routes	Every route under /api/persons*, /api/staff/*, /api/admin/* now calls getFakeUserFromToken() and returns 401 Unauthorized if the token is missing/invalid	Removes the "no token = defaults to user 1 / full dataset" behavior
Fix 7 — Owner-based access control	GET/PUT/DELETE /api/persons/{id} compares the JWT's user_id against the record's user_id; mismatched non-staff/admin requests get 403	Closes Test 2 (BOLA)
Fix 8 — Role-based access control (RBAC)	/api/staff/* requires role ∈ {staff, admin}; /api/admin/* requires role === admin; otherwise 403 Forbidden	Closes Test 3 and Test 4 (BFLA)
Fix 9 — Mass assignment prevention	PUT /api/persons/{id} whitelists only name, age, height, weight, notes; user_id, role, bmi, category from the request body are ignored, and bmi/category are recalculated server-side after the update	Closes Test 8
Fix 10 — Sensitive data removal	register, login, profile, and admin/users now SELECT id, name, email, role, created_at only — never password or password_hash	Closes Test 6
Fix 11 — XSS fix	BmiCard.vue: → <p v-text="person.notes">	Closes Test 5 — notes render as literal text, scripts/HTML never execute
Fix 12 — Generic error handling + remaining prepared statements	exposeException() now logs the full exception (error_log) server-side and returns only {"error": "Unable to process	Stops leaking file paths/line numbers/stack traces to API clients

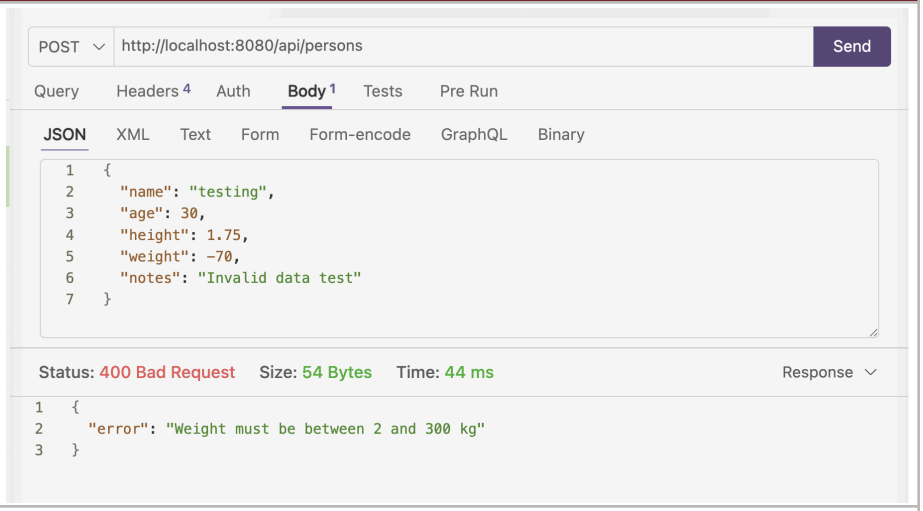
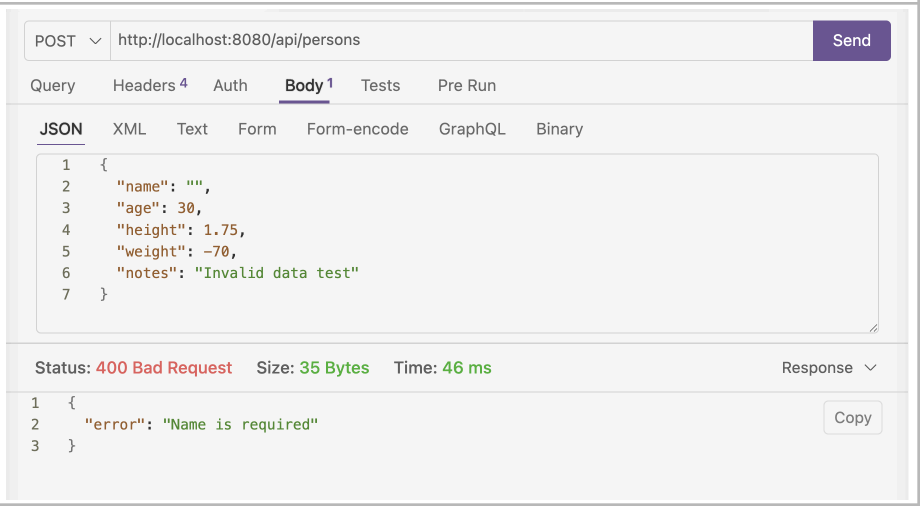
Fix	What changed	How it addresses the weakness
	request"} (500) to the client; login query fully parameterized	

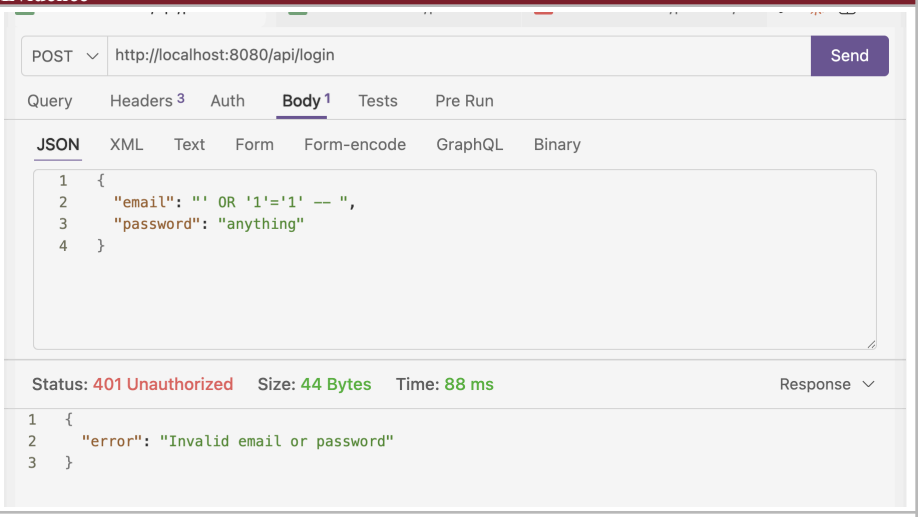
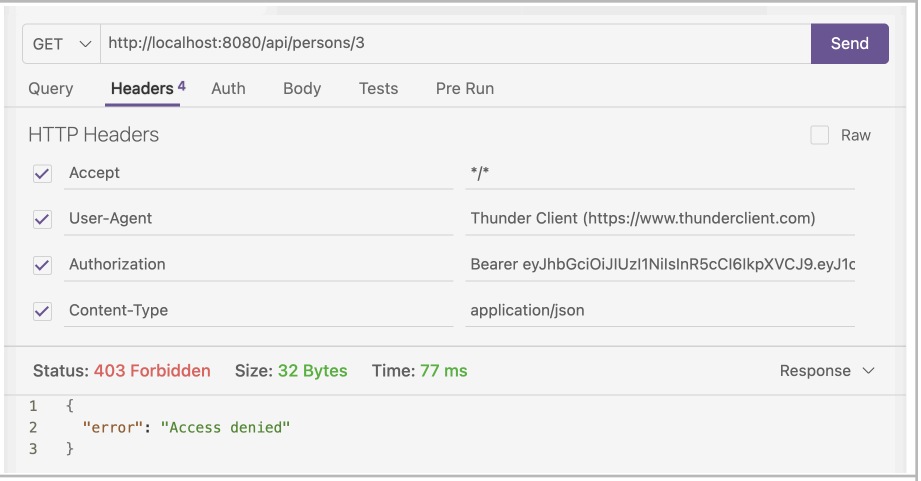
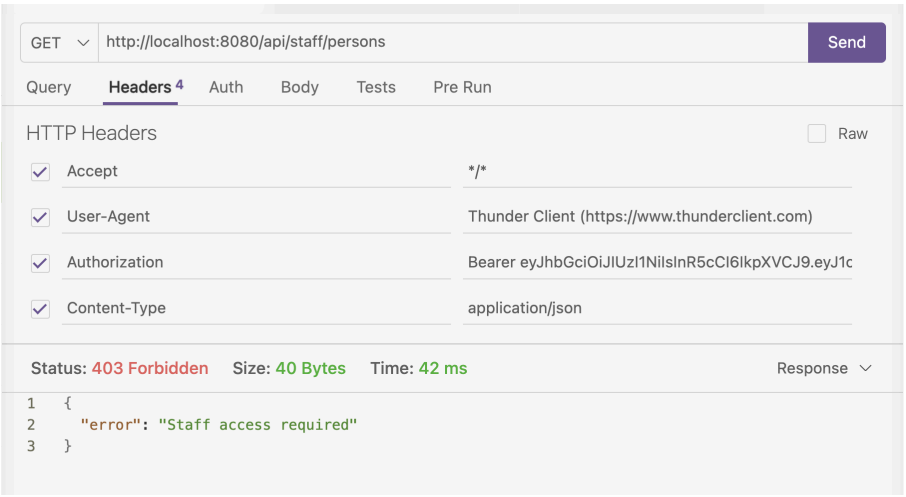
Problem-solution mapping table.

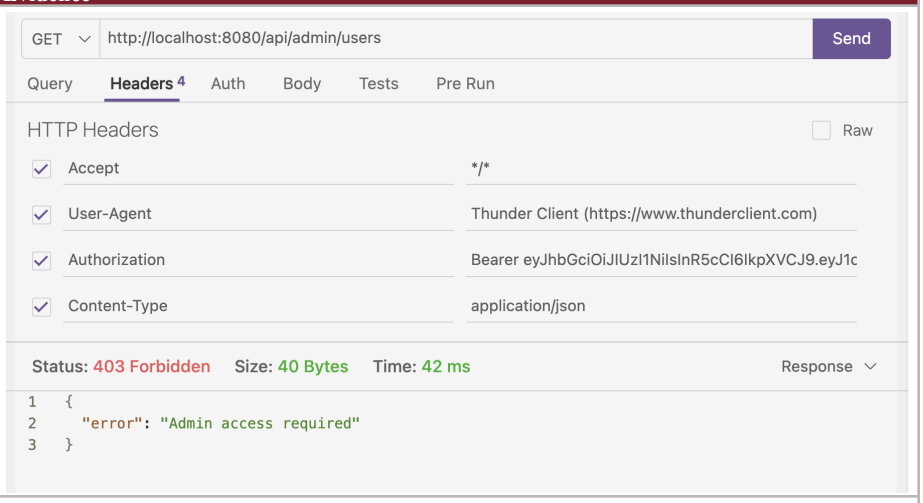
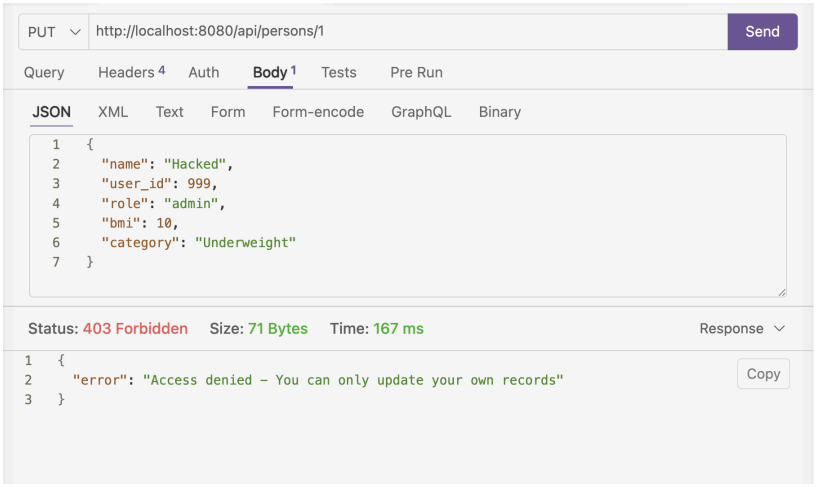
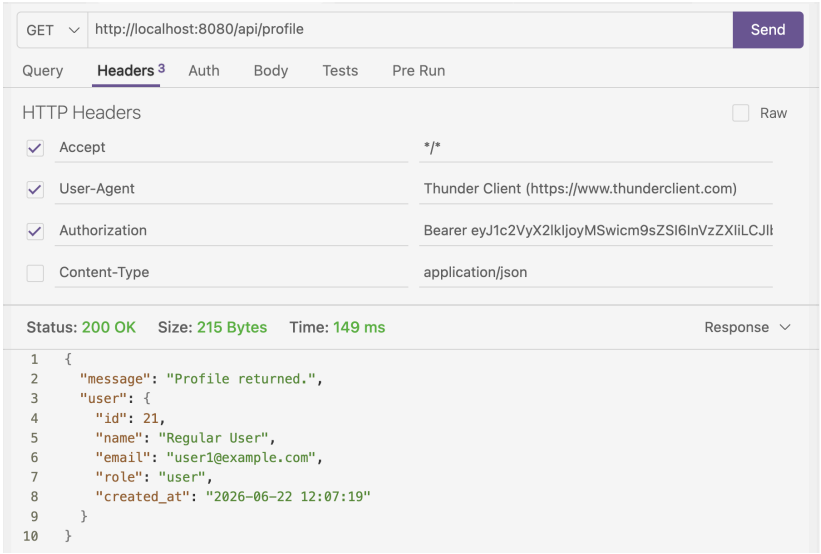
Problem	Root Cause	Where to Fix?	Proposed Solution	How to Retest?
Invalid BMI accepted	Backend does not validate	Backend	Validate name, age, height, weight	Submit invalid data again
User accesses other record	No ownership check	Backend API	Compare JWT user_id with record user_id	Access other user's record
User accesses admin route	No role check	Backend API	Check role from verified JWT	Use user token on admin route
password_hash returned	API returns all fields	Backend API	Select only safe fields	Check JSON response
XSS executes	Unsafe rendering	Frontend	Use {{ }} or textContent	Submit XSS payload again
SQL Injection possible	String SQL query	Backend	Use prepared statement	Try injection input again
user_id can be changed	Blind update	Backend API	Allow only safe fields	Send user_id in PUT request

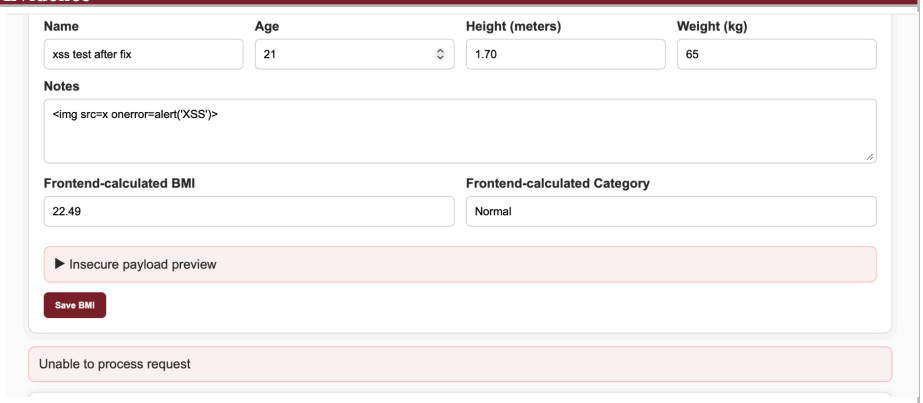
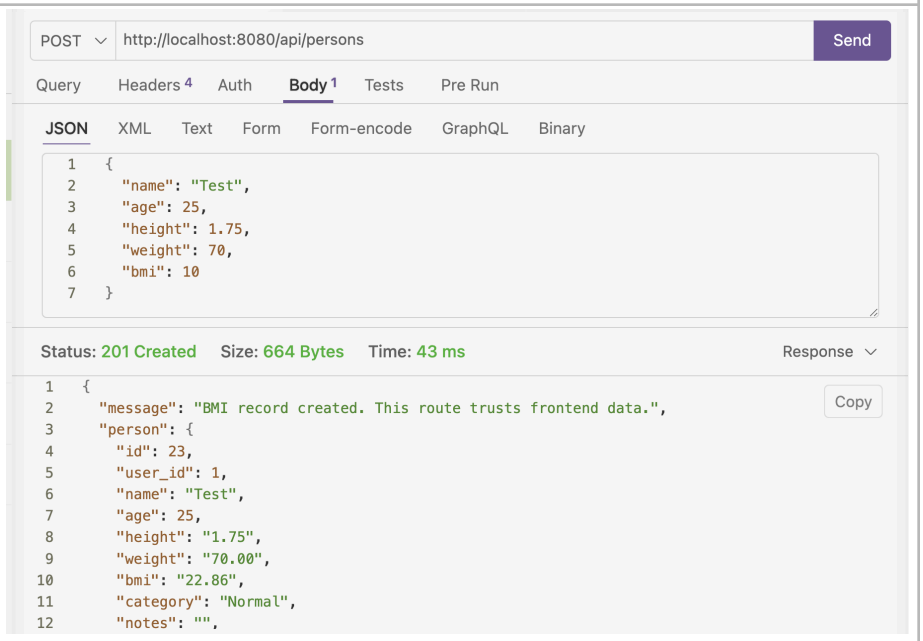
5. Testing evidence

Before-and-after testing evidence.

Test Case	Before Fix	After Fix	Evidence
Add BMI with negative weight	Accepted	400 Weight must be between 2 and 300 kg	
Submit empty name	Accepted	400 Name is required	

Test Case	Before Fix	After Fix	Evidence
SQL Injection in login input	Logged in successfully	Invalid Login	
Access another user's BMI record	Data returned	403 Access denied	
Access staff route as user	Allowed	403 Staff access required	

Test Case	Before Fix	After Fix	Evidence
Access admin route as user	Allowed	403 Admin access required	
Update user_id or role	Accepted	Rejected or ignored	
API returns password_hash	Visible	Removed	

Test Case	Before Fix	After Fix	Evidence
XSS payload in notes	Alert executes	Displayed as text	
Submit with BMI value	Fake value accepted	Returns correct bmi	

6. API endpoints tested

Method	Endpoint	Purpose	Security Control Verified
POST	/api/register	Create user	Prepared statements, no sensitive fields returned
POST	/api/login	Authenticate	SQLi resistance, JWT issuance, no sensitive fields returned
GET	/api/profile	Get own profile	JWT required, no password hash in response
GET	/api/persons	List own/all BMI records	JWT required, ownership scoping
POST	/api/persons	Create BMI record	Input validation, backend BMI calculation
GET	/api/persons/{id}	Read one record	Ownership check (BOLA)
PUT	/api/persons/{id}	Update record	Ownership check + field whitelist (mass assignment)
DELETE	/api/persons/{id}	Delete record	Ownership/role check
GET	/api/staff/persons	Staff view of all records	RBAC (staff/admin only)
GET	/api/staff/persons/{id}	Staff view of one record	RBAC
GET	/api/admin/users	List all users	RBAC (admin only), no sensitive fields
PUT	/api/admin/users/{id}/role	Change a user's role	RBAC
DELETE	/api/admin/persons/{id}	Admin delete	RBAC

7. Reflection answers

Which weakness was easiest to find? Why?

The negative/invalid BMI input (Test 1) and the exposed password_hash (Test 6) were the easiest because both were visible immediately in the raw JSON response with no special tooling, just sending a normal request and reading the body.

Which weakness was most dangerous? Why?

The SQL Injection login bypass (Test 7). It defeats authentication entirely because an attacker doesn't need a valid account at all, just a crafted email field, and gets a working session token. It also sits upstream of every other control: once in, BOLA/BFLA fixes are moot if the attacker can log in as anyone.

Why is frontend validation not enough?

Frontend code runs entirely on the client's machine. Anyone can bypass the UI and call the API directly with Postman, curl, or a modified script which is exactly how Test 1 and Test 8 were performed. Frontend validation is a UX convenience, not a security boundary; only checks enforced on the server, where the attacker has no control, are trustworthy.

Why should BMI calculation be performed at the backend?

If the client sends bmi and category directly, a user can submit any values they want, for example: claim "Normal" while sending an obviously unhealthy height/weight pair which will corrupt the dataset's integrity. Calculating BMI server-side (Fix 2) guarantees the stored value is always mathematically derived from the validated height/weight, regardless of what the client claims.

What is the difference between authentication and authorization?

Authentication verify identity, done here via /api/login issuing a JWT. Authorization is when given a verified identity, authorization decides whether that identity can access a specific resource or action, done here via the ownership checks (Fix 7) and role checks (Fix 8). The original app had neither working correctly: a forged/unsigned token meant authentication was meaningless, and missing checks meant authorization didn't exist at all.

What is the difference between RBAC and owner-based access control?

RBAC (Fix 8) grants access based on a user's role, for example: anyone with role: staff or role: admin can see all BMI records, regardless of whose records they are. Owner-based access control (Fix 7) grants access based on relationship to the specific resource — a regular user can only access a record where record.user_id == token.user_id, independent of role. Both are needed: RBAC handles "which whole routes can this role reach," ownership handles "within a route, which specific records can this individual reach."

Why is JWT alone not enough to protect all data?

A valid JWT only proves who the requester is, it says nothing about whether that identity is allowed to touch a particular record or route. Before Fix 7/8, the app validated the token (Fix 6) but still let any authenticated user read any other user's BMI record or hit staff/admin routes. Authorization logic has to be checked explicitly, every time, on top of a valid token.

Why should API responses exclude password_hash?

Even though it's hashed rather than plaintext, a leaked hash is still an offline attack target — password_hash was found to literally contain "password123" in plaintext during testing because the password-hashing fix hadn't been wired into the response-building code at the time. More generally, returning any credential material gives attackers a target to crack offline (rainbow tables, brute force) with no rate limiting. The fix (Fix 10) is to explicitly SELECT only the safe fields the client actually needs, never SELECT *.

Why is a Vue route guard not enough to protect backend routes?

A frontend route guard only stops the Vue Router from rendering a page in the browser, it doesn't touch the API at all. An attacker who knows or guesses an endpoint can call it directly with Postman/curl, completely bypassing the Vue app and its route guards, exactly as demonstrated in Tests 3 and 4. Every protection visible in the UI must be re-enforced independently on the server, because the server is the only component the attacker cannot directly control.

How did you prove that your security fix works?

For each weakness, the same request that previously succeeded data was re-sent after the fix and the response was captured: invalid BMI input now returns 400; cross-user record access now returns 403; staff/admin routes return 403 for a user-role token; the SQLi payload now returns 401 Invalid login instead of bypassing auth; updating user_id/role/bmi via PUT no longer changes those fields; password_hash no longer appears in any JSON response; and the XSS payload renders as visible text instead of firing an alert().